

Real-world (persist and read back real data) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Every version below creates `runs.db` in the current folder the first time it runs. Delete that file between tries if you want a clean slate. Read all three depths — they're not "wrong, less wrong, right," they're 3 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# sqlite_persistence_practice.py – Real-world section
import sqlite3
from datetime import datetime, timezone
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
conn = sqlite3.connect("runs.db", check_same_thread=False)
conn.execute("""
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT,
        output TEXT,
        created_at TEXT
    )
""")

class ChatRequest(BaseModel):
    message: str

@app.post("/chat")
def chat(request: ChatRequest):
    reply = "You said: " + request.message
    now = datetime.now(timezone.utc).isoformat()
    conn.execute(
        "INSERT INTO runs (input, output, created_at) VALUES (?, ?, ?)",
```

```

        (request.message, reply, now),
    )
    conn.commit()
    return {"reply": reply}

@app.get("/runs")
def get_runs():
    rows = conn.execute("SELECT * FROM runs").fetchall()
    return rows

```

Expected output (`POST /chat` with `{"message": "hi"}` , then `GET /runs`):

```

POST /chat -> {"reply": "You said: hi"}
GET /runs -> [[1, "hi", "You said: hi", "2026-01-01T00:00:00.000000+00:00"]]

```

This saves and reads back real data — the core requirement — but `GET /runs` returns unlabeled tuples, which is technically valid JSON but not something a real client could use without guessing field order.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — `sqlite3.Row`, real dicts back out

```
# sqlite_persistence_practice.py – Real-world section
import sqlite3
from datetime import datetime, timezone
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
conn = sqlite3.connect("runs.db", check_same_thread=False)
conn.row_factory = sqlite3.Row
conn.execute("""
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT,
        output TEXT,
        created_at TEXT
    )
""")

class ChatRequest(BaseModel):
    message: str

@app.post("/chat")
def chat(request: ChatRequest):
    reply = "You said: " + request.message
    now = datetime.now(timezone.utc).isoformat()
    conn.execute(
        "INSERT INTO runs (input, output, created_at) VALUES (?, ?, ?)",
        (request.message, reply, now),
    )
    conn.commit()
    return {"reply": reply}

@app.get("/runs")
def get_runs():
    rows = conn.execute("SELECT * FROM runs").fetchall()
    return [dict(row) for row in rows]
```

Expected output (GET /runs):

```
[{"id":1,"input":"hi","output":"You said: hi","created_at":"2026-01-01T00:00:00.000000"}]
```

Approach 2 — a small `db.py` helper instead of a bare module-level `conn`

```
# sqlite_persistence_practice.py – Real-world section
import sqlite3

def get_connection():
    conn = sqlite3.connect("runs.db", check_same_thread=False)
    conn.row_factory = sqlite3.Row
    conn.execute("""
        CREATE TABLE IF NOT EXISTS runs (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            input TEXT,
            output TEXT,
            created_at TEXT
        )
    """)
    return conn

def insert_run(conn, input_text, output_text, created_at):
    conn.execute(
        "INSERT INTO runs (input, output, created_at) VALUES (?, ?, ?)",
        (input_text, output_text, created_at),
    )
    conn.commit()

def fetch_all_runs(conn):
    rows = conn.execute("SELECT * FROM runs").fetchall()
    return [dict(row) for row in rows]
```

```
# sqlite_persistence_practice.py – Real-world section
from datetime import datetime, timezone
from fastapi import FastAPI
from pydantic import BaseModel
from db import get_connection, insert_run, fetch_all_runs

app = FastAPI()
conn = get_connection()

class ChatRequest(BaseModel):
    message: str
```

```

@app.post("/chat")
def chat(request: ChatRequest):
    reply = "You said: " + request.message
    now = datetime.now(timezone.utc).isoformat()
    insert_run(conn, request.message, reply, now)
    return {"reply": reply}

@app.get("/runs")
def get_runs():
    return fetch_all_runs(conn)

```

Expected output: identical to Approach 1 — this only reorganizes the code, matching the `db/database.py` / `db/models.py` split the Build Task's `Suggested files` section uses.

Difference from Basic: both Intermediate approaches add `conn.row_factory = sqlite3.Row` and convert each row with `dict(row)`, so `GET /runs` returns real, field-named JSON instead of positional tuples a client would have to guess the order of. Approach 2 additionally moves the SQL into a dedicated `db.py`, keeping `main.py` focused on routing — the same separation-of-concerns move `chat_route`'s Approach 2 made for its Pydantic models.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a typed `response_model`, and a real `GET /runs/{id}` with a 404

```
# sqlite_persistence_practice.py – Real-world section
import sqlite3
from datetime import datetime, timezone
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()
conn = sqlite3.connect("runs.db", check_same_thread=False)
conn.row_factory = sqlite3.Row
conn.execute("""
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT,
        output TEXT,
        created_at TEXT
    )
""")

class ChatRequest(BaseModel):
    message: str

class RunRecord(BaseModel):
    id: int
    input: str
    output: str
    created_at: str

@app.post("/chat")
def chat(request: ChatRequest):
    reply = "You said: " + request.message
    now = datetime.now(timezone.utc).isoformat()
    conn.execute(
        "INSERT INTO runs (input, output, created_at) VALUES (?, ?, ?)",
        (request.message, reply, now),
    )
    conn.commit()
    return {"reply": reply}
```

```

@app.get("/runs", response_model=list[RunRecord])
def get_runs():
    rows = conn.execute("SELECT * FROM runs").fetchall()
    return [dict(row) for row in rows]

@app.get("/runs/{run_id}", response_model=RunRecord)
def get_run(run_id: int):
    row = conn.execute("SELECT * FROM runs WHERE id = ?", (run_id,)).fetchone()
    if row is None:
        raise HTTPException(status_code=404, detail="run not found")
    return dict(row)

```

Expected output for `GET /runs/1` (existing row):

```
200 {"id":1,"input":"hi","output":"You said: hi","created_at":"2026-01-01T00:00:00"}
```

Expected output for `GET /runs/999` (no such row):

```
404 {"detail":"run not found"}
```

Note the `?` placeholder and `(run_id,)` tuple — even with a single value, it still has to be passed as a tuple, and it's still never string-formatted directly into the SQL.

Approach 2 — one shared connection via FastAPI's lifespan, not a bare module-level global

Opening the connection as a bare module-level `conn = sqlite3.connect(...)` (every approach above) works for this exercise, but ties database setup to *import time*, which makes it awkward to test (you can't easily swap in a test database) and doesn't give you a clean shutdown hook.

FastAPI's `lifespan` runs setup once at startup and teardown once at shutdown.

```

# sqlite_persistence_practice.py – Real-world section
import sqlite3
from contextlib import asynccontextmanager
from datetime import datetime, timezone
from fastapi import FastAPI, HTTPException, Request
from pydantic import BaseModel

class RunRecord(BaseModel):
    id: int
    input: str
    output: str
    created_at: str

```

```

class ChatRequest(BaseModel):
    message: str

@asynccontextmanager
async def lifespan(app: FastAPI):
    conn = sqlite3.connect("runs.db", check_same_thread=False)
    conn.row_factory = sqlite3.Row
    conn.execute("""
        CREATE TABLE IF NOT EXISTS runs (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            input TEXT,
            output TEXT,
            created_at TEXT
        )
    """)
    app.state.db = conn
    yield
    conn.close()

app = FastAPI(lifespan=lifespan)

@app.post("/chat")
def chat(request: ChatRequest, http_request: Request):
    conn = http_request.app.state.db
    reply = "You said: " + request.message
    now = datetime.now(timezone.utc).isoformat()
    conn.execute(
        "INSERT INTO runs (input, output, created_at) VALUES (?, ?, ?)",
        (request.message, reply, now),
    )
    conn.commit()
    return {"reply": reply}

@app.get("/runs/{run_id}", response_model=RunRecord)
def get_run(run_id: int, http_request: Request):
    conn = http_request.app.state.db
    row = conn.execute("SELECT * FROM runs WHERE id = ?", (run_id,)).fetchone()
    if row is None:
        raise HTTPException(status_code=404, detail="run not found")
    return dict(row)

```

Expected output: identical to Approach 1 — the connection now lives on `app.state.db`, opened once at startup and closed once at shutdown, instead of a bare global created the instant the module is imported.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate returns usable JSON but only ever hands back the *whole* table. Approach 1 adds a typed `RunRecord` response model (validated on the way out, same pattern as `chat_route`'s `ChatResponse`) and a genuine single-row lookup that handles "not found" as a clean 404 instead of crashing or returning `null`. Approach 2 doesn't change the SQL at all — it moves connection setup out of import-time and into FastAPI's `lifespan`, which is what actually lets you swap in a separate test database later without editing route code.

Which one should you actually write? Intermediate Approach 1 (`row_factory` + `dict(row)`) is the minimum any SQLite-backed route needs — never return a raw `sqlite3.Row` or tuple from a route. Add Advanced Approach 1's typed `response_model` and `GET /runs/{id}` as soon as a client needs to look up one specific run, which the Build Task's requirements explicitly ask for. Reach for Approach 2's `lifespan` pattern once you're writing tests for this (a fresh `:memory:` database per test is the standard move) or once you have real startup/shutdown work beyond just `CREATE TABLE`.